Database Per Microservice

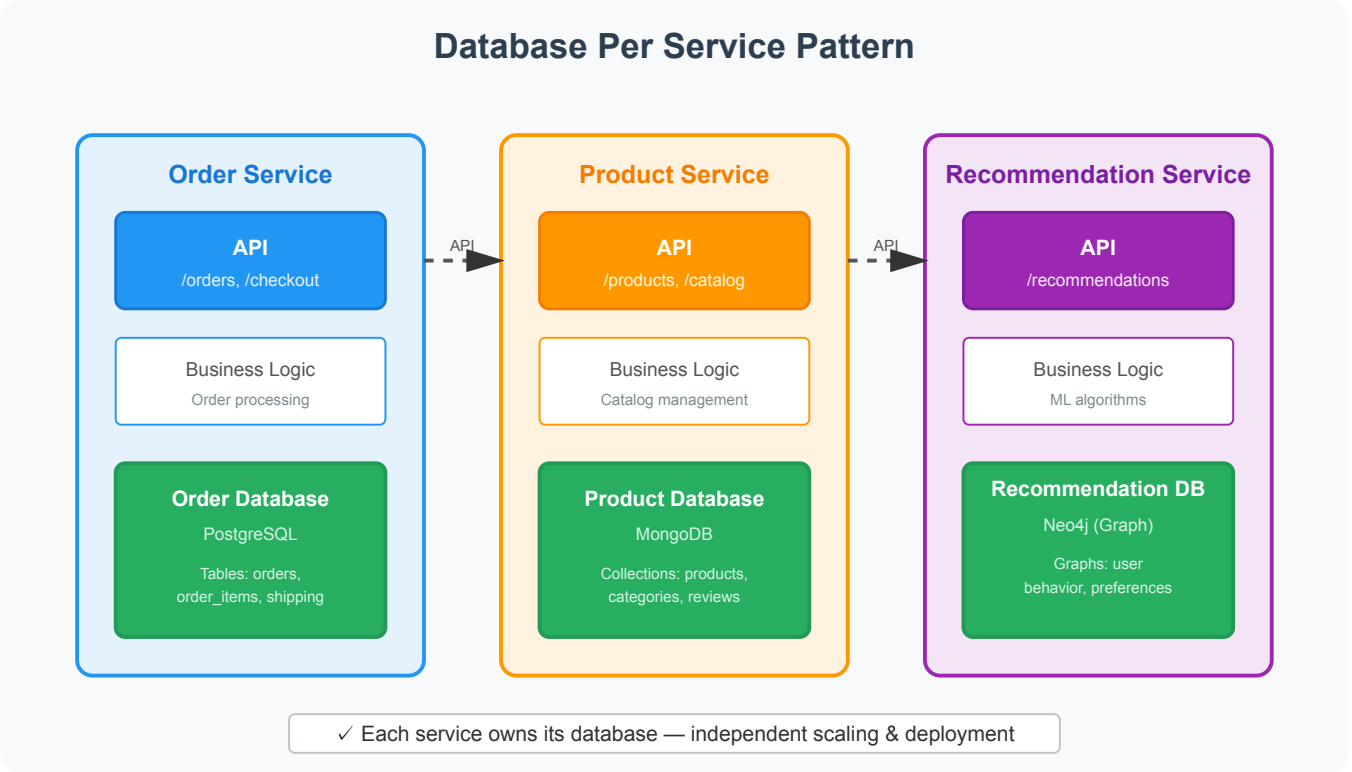
systemdesignschool.io/fundamentals/a-database-per-service

Caching reduces database load by serving repeated reads from memory. But scaling writes requires a different approach. The database-per-service pattern assigns each microservice its own database to enable independent scaling and deployment.

A monolithic application uses one large database. All components share tables. This coupling creates problems. A schema change in one module breaks others. One slow query affects everyone. Scaling requires scaling the entire database, even when only one module needs more capacity.

How It Works

Each microservice owns its database completely. No service accesses another service's database directly. Consider an e-commerce platform as an example. Order Service manages order tables. Product Service manages catalog tables. Recommendation Service manages user behavior data.



When Order Service needs product information, it calls Product Service's API. Product Service queries its own database and returns results. This API boundary enforces isolation. Schema changes in Product Service stay contained. The Order Service only sees the API contract, not the database schema.

Benefits

Independent scaling becomes possible. For example, the Recommendation Service runs complex analytics queries requiring high CPU and memory. It uses a dedicated database server with 64 cores and 256GB RAM. The Order Service handles simple CRUD operations. It runs on smaller servers with SSDs for fast writes. Each service scales based on its needs instead of shared requirements.

Technology flexibility follows from isolation. The Order Service uses PostgreSQL for transactional consistency. The Product Service uses MongoDB for flexible product schemas. The Recommendation Service uses Neo4j for graph queries. Teams choose databases matching their data patterns without coordination.

Schema changes deploy independently. The Product Service team adds a new column for product dimensions. They deploy the schema change and update their service code. Other services continue working unchanged. No coordination meetings. No cross-team dependencies. Changes ship faster.

Clear ownership improves maintainability. The Product Service team owns their database schema, queries, indexes, and backups. They understand query patterns deeply. When performance degrades, one team investigates without involving others.

Challenges

Data consistency across services requires careful design. In a monolithic database, foreign key constraints enforce consistency. An order references a product. The database prevents deleting a product while orders reference it. With separate databases, this protection disappears.

Application logic must handle consistency. Before deleting a product, check if any orders reference it. This requires calling Order Service. The check and delete cannot happen atomically across services. Another order might create a reference between the check and the delete. Solutions include saga patterns, eventual consistency, or accepting temporary inconsistency.

Querying across services becomes complex. A report showing orders with product details requires data from both services. The application must call Order Service for orders, extract product IDs, call Product Service for details, and join results. This amplifies latency and code complexity compared to a SQL join.

Operational overhead increases. One database requires one backup system, one monitoring setup, one upgrade process. Five services with five databases require five of everything. Operations teams face more complexity managing multiple database technologies.

When to Use

Use database-per-service when building microservices with distinct scaling needs, different data access patterns, or independent deployment requirements. The pattern applies across domains. E-commerce platforms separate order processing, catalog browsing, and analytics. Social media platforms separate user profiles, content feeds, and messaging. Video streaming platforms separate user accounts, content metadata, and viewing history. Each service scales independently based on its traffic patterns.

Use separate databases when teams own different services and need autonomy. A large organization with 20 teams building different features cannot coordinate every schema change. Database isolation lets teams move independently.

Avoid database-per-service for small systems or early-stage startups. The operational overhead outweighs benefits when three developers manage the entire system. Start with a monolithic database. Split when scaling demands or team size justifies the complexity.

Interview Application

When designing a system with microservices, explain database isolation by service. For an e-commerce system, Order Service, Product Service, and Inventory Service each own their databases. For a social network, User Service, Post Service, and Notification Service separate their data. This enables independent scaling. Each service optimizes for its access patterns.

Discuss the consistency trade-off. When operations span services, they cannot use a single transaction. For e-commerce checkout, the system must decrement inventory, charge payment, and create an order across three databases. Explain using the saga pattern with compensating transactions for coordination.

Mention operational cost. Five services mean five databases to backup, monitor, and upgrade. Acknowledge this complexity but explain it's worthwhile at scale when teams work independently and services have different requirements.

Show awareness that small systems should not use this pattern. For a system with 100,000 users and 3 developers, the overhead is not justified. The pattern makes sense at larger scale with multiple teams.